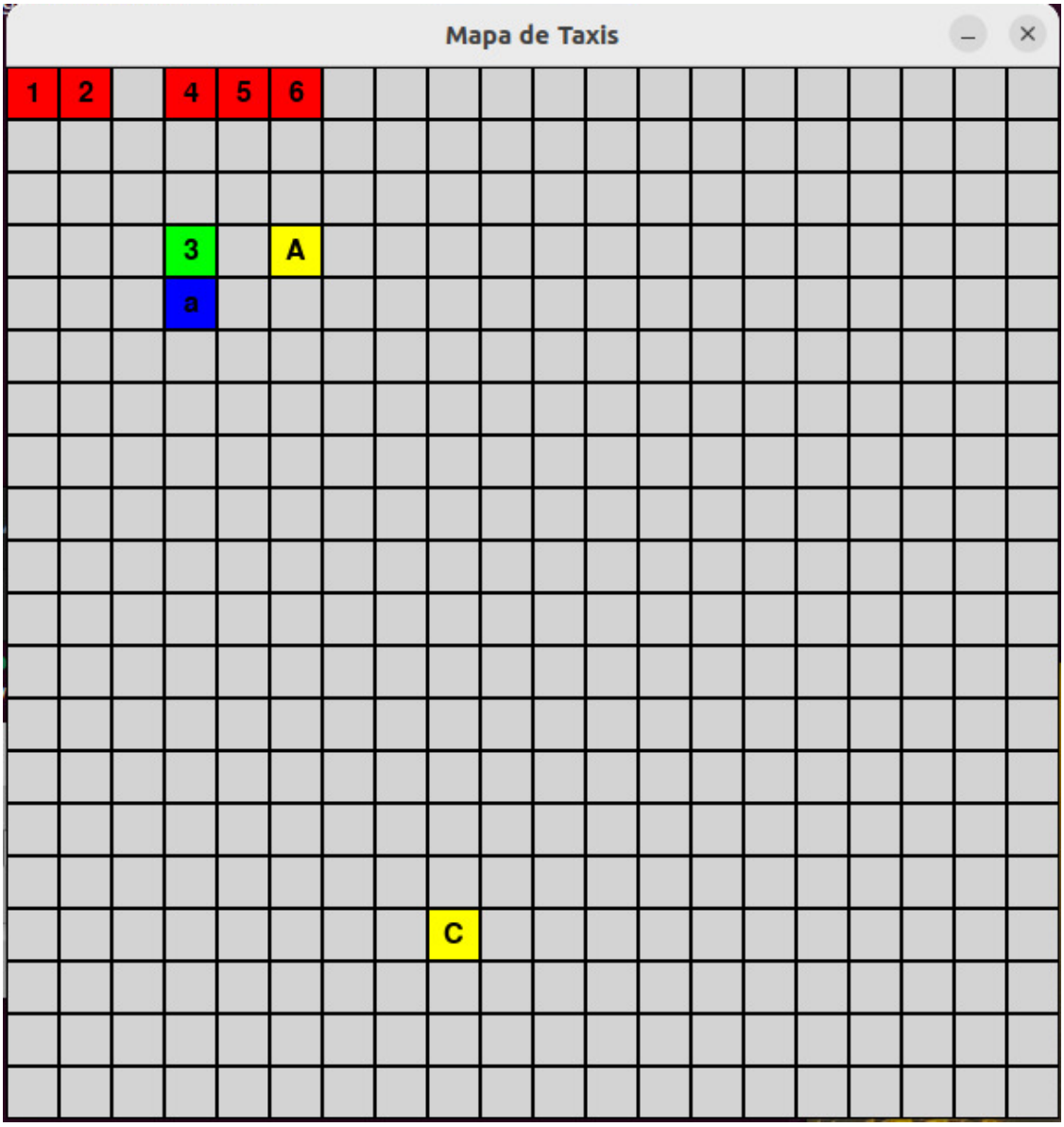
SISTIMAS DISTRIBUIDOS

PRÁCTICA 3
EASY CAB RELEASE



Aitana Cantó Rico, Levan Kipiani
GRUP0 1 TEO, 02 - 2 PRÁCTICAS

ÍNDICE

- **CENTRAL** 2
- **EC_DE (Digital Engine)** 5
- **EC_CTC (City Traffic Control)** 7
- **EC_Registry** 9
- **EC_API** 11
- **FRONT-END** 13
- **SENSOR** 15
- **CUSTOMER** 16
- **MAPA PYGAME** 18
- **GUÍA DE DESPLIEGUE** 19
- **CONTROL DE VERSIONES** 22

INTRODUCCIÓN

Nuestro sistema EasyCab es un sistema distribuido diseñado para gestionar servicios de transporte urbano mediante la coordinación eficiente de taxis, clientes y módulos auxiliares. Basado en una arquitectura SOA, utiliza tecnologías como Kafka, APIs REST y sockets para facilitar la comunicación entre componentes.

El módulo central, EC_Central, administra solicitudes de taxis, asigna vehículos y coordina rutas en un mapa interactivo actualizado en tiempo real. Este mapa refleja las posiciones de taxis, clientes y destinos, y se integra con visualizadores como Pygame y el módulo EC_API.

EasyCab incorpora EC_CTC, que utiliza la API de OpenWeather para ajustar operaciones según el estado del tráfico, activando emergencias si es necesario. Además, el módulo EC_Registry gestiona el registro y la operatividad de los taxis.

La seguridad es fundamental: cada taxi emplea cifrado simétrico para proteger las comunicaciones con EC_Central, reduciendo riesgos de interceptación. También incluye EC_Sensor, que controla permisos de movimiento en tiempo real mediante señales específicas.

Con un diseño modular y escalable, EasyCab es adaptable a diversos entornos urbanos y demuestra la aplicación de principios avanzados de sistemas distribuidos para resolver problemas reales de logística urbana.

○ CENTRAL

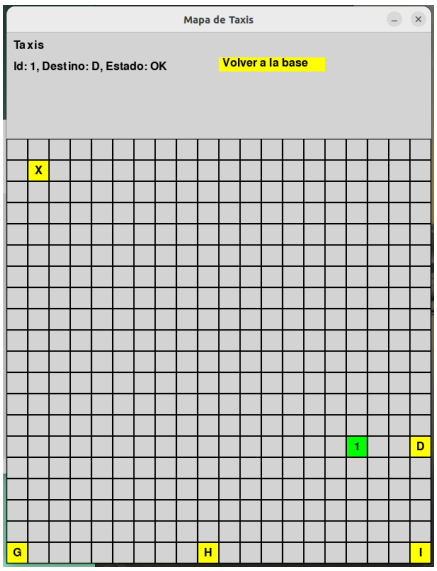
• Descripción del Módulo

El módulo **EC_Central** es el núcleo del sistema EasyCab, encargado de gestionar y coordinar las operaciones de taxis y clientes, así como de interactuar con otros módulos del sistema. Con la nueva versión de la práctica, se han añadido funcionalidades de seguridad, integración con el tráfico de la ciudad, y auditoría de eventos.

Este módulo utiliza Kafka para gestionar la comunicación entre taxis y clientes, y sockets para recibir conexiones de los taxis. No interactúa directamente con los usuarios, ya que es un backend que coordina la lógica interna del sistema, incluyendo la actualización del mapa interactivo en mapaPygame y el módulo EC_API.

• Funcionalidades Principales

1. **Gestión de Solicitudes de Taxis**:
    - Procesa las solicitudes de clientes y asigna taxis disponibles.
    - Coloca en espera las solicitudes si no hay taxis disponibles.
    - Coordina las rutas de los taxis hacia los destinos especificados.
2. **Integración con EC_CTC (City Traffic Control)**:
    - Consume la API de tráfico de EC_CTC para determinar si el estado del tráfico es "OK" o "KO".
    - En caso de "KO", activa el estado de emergencia y ordena a todos los taxis que vuelvan a la base (posición inicial).
3. **Cifrado y Seguridad**:
    - Implementa un sistema de cifrado simétrico utilizando la biblioteca cryptography.fernet.
    - Cifra las órdenes de viaje y las comunicaciones hacia los taxis.
    - Valida tokens únicos generados para cada taxi durante la autenticación inicial.
4. **Auditoría de Eventos**:
    - Registra eventos relevantes en un archivo de auditoría, incluyendo cambios de estado del tráfico, autenticaciones fallidas y operaciones críticas.
5. **Mapa Interactivo**:
    - Representado como una matriz de 20x20 que actualiza continuamente la ubicación de taxis, clientes y destinos.
    - Mantiene y actualiza el estado del mapa en tiempo real.
    - Envía las actualizaciones al módulo EC_API para su visualización en tiempo real y al visualizador Pygame.



• Detalles de Implementación

1. **Estado del Tráfico**:
    - Consulta el estado de tráfico cada 10 segundos a través de la API de EC_CTC.
    - Si el tráfico no es viable ("KO"), activa el estado de emergencia y ordena a los taxis regresar a la base (posición inicial).
    - Registra los cambios de estado del tráfico en los logs de auditoría.

2. **Cifrado de Comunicaciones (Simétrico)**:
    - **Generación de Claves**: Cada taxi tiene una clave única generada con Fernet.generate_key() durante su registro. Estas claves se almacenan en un archivo seguro (claves.txt).
    - **Cifrado de Órdenes**:
        - Antes de enviar una orden al taxi, el mensaje en formato JSON es serializado y convertido a bytes.
        - Se utiliza la clave específica del taxi para cifrar el mensaje mediante la función encrypt_message(), que aplica el método Fernet(key).encrypt().
        - El mensaje cifrado es enviado al taxi a través de Kafka.
    - **Descifrado de Mensajes**:
        - Los mensajes recibidos de los taxis están cifrados.
        - EC_Central utiliza la clave asociada al taxi para descifrar el contenido con Fernet(key).decrypt().
        - Una vez descifrado, el contenido se procesa como un mensaje JSON estándar.
3. **Gestión de Tokens**:
    - Genera tokens únicos para autenticar a los taxis durante el registro.
    - Valida los tokens con cada mensaje recibido de los taxis para evitar accesos no autorizados.
4. **Auditoría de Eventos**:
    - Los eventos auditados incluyen:
        - Autenticaciones exitosas y fallidas.
        - Cambios en el estado del tráfico.
        - Órdenes de emergencia.
    - Se registra la IP de origen, el tipo de evento y los detalles relacionados.
5. **Mapa y Coordinación de Taxis**:
    - Gestiona el estado del mapa como una matriz de 20x20.
    - Actualiza el mapa en tiempo real con posiciones de taxis, clientes y destinos.
    - Coordina los movimientos de los taxis hacia los destinos especificados.
6. **Conexión Kafka**:
    - Utiliza los siguientes topics:
        - solicitud_taxi: Para recibir peticiones de los clientes.

- procesar_viaje: Para enviar órdenes cifradas a los taxis.
- actualizar_mapa: Para enviar actualizaciones del estado del mapa.

7. **Conexión Sockets**:
   - Valida y autentica taxis mediante sockets en startTaxi.
   - Cada taxi tiene un hilo independiente para su verificación y estado.

8. **Claves y archivos necesarios**:
   - claves.txt: Para el almacenamiento de claves de cifrado.
   - token.txt: Para los tokens de autenticación.
   - audit_log.txt: Archivo de auditoría.
   - mapa.txt: Configuración inicial del mapa.
   - taxi.txt: Estado inicial de los taxis.

○ EC_DE (Digital Engine)

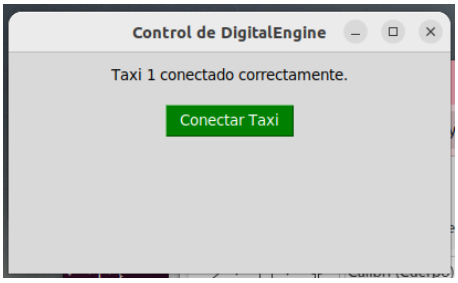
• Descripción del Módulo

El módulo **EC_DE (Digital Engine)** actúa como el "motor digital" del sistema EasyCab. Su función principal es gestionar los viajes de los taxis, desde recoger a los clientes hasta llevarlos a su destino, mientras se comunica con otros módulos del sistema, como **EC_Central** y el sensor (EC_S). Con la nueva versión, este módulo también implementa cifrado de comunicaciones, integración con el registro de taxis (EC_Registry) y un sistema de estado de emergencia.

• Funcionalidades Principales

1. **Gestor de Viajes**:
   - Recibe órdenes de viaje de **EC_Central**.
   - Mueve el taxi hacia el cliente y luego al destino.
   - Informa sobre la posición actual y el estado del viaje.
2. **Estado de Emergencia**:
   - Responde a órdenes de emergencia enviadas por **EC_Central**.
   - Devuelve el taxi a la base en caso de estado "KO".

3. **Cifrado y Seguridad**:
    - Cifra los mensajes enviados a **EC_Central** usando una clave simétrica específica para cada taxi.
    - Descifra las órdenes recibidas de **EC_Central**.
4. **Registro en EC_Registry**:
    - Registra y actualiza el estado del taxi en el módulo EC_Registry.
5. **Integración con el Sensor**:
    - Se comunica con el sensor para obtener permisos de movimiento (OK/KO/FIN).
6. **Interfaz Gráfica**:
    - Proporciona controles para conectar y desconectar el taxi, mostrando el estado actual del taxi.



• Detalles de Implementación

1. **Cifrado de Comunicaciones (Simétrico)**:
    - **Generación de Clave**: Cada taxi recibe una clave simétrica de **EC_Central** durante la autenticación inicial.
    - **Cifrado de Mensajes**:
        - Los mensajes que incluyen la posición y el estado del taxi se cifran con Fernet.encrypt() usando la clave asignada.
        - El mensaje cifrado se envía a **EC_Central** mediante Kafka.
    - **Descifrado de Órdenes**:
        - Las órdenes de viaje recibidas se descifran usando Fernet.decrypt() antes de procesarlas.
2. **Gestor de Viajes**:
    - **Movimiento**:
        - Utiliza la función movimiento() para calcular el desplazamiento hacia el destino, considerando bloqueos y caminos alternativos.
    - **Flujo del Viaje**:
        - Recoge al cliente y lo lleva al destino.
        - Envía actualizaciones de estado y posición a **EC_Central**.

3. **Estado de Emergencia**:
    - Monitorea constantemente el estado del sistema mediante Kafka.
    - En caso de emergencia, interrumpe el viaje actual y lleva el taxi de vuelta a la base.
4. **Conexión Kafka**:
    - **Topics utilizados**:
        - procesar_viaje: Recibe órdenes de viaje y de emergencia de **EC_Central**.
        - actualizar_mapa: Recibe actualizaciones del estado del mapa.
        - actualizaciones_taxi: Envía las actualizaciones de posición y estado del taxi a **EC_Central**.
5. **Conexión con el Sensor**:
    - Establece una conexión mediante sockets para recibir permisos de movimiento.
    - Responde a los comandos OK, KO y FIN según las señales del sensor.
6. **Registro en EC_Registry**:
    - Se registra al inicio en EC_Registry mediante una petición HTTP al endpoint /register_taxi.
    - Actualiza su estado en caso de desconexión.

○ EC_CTC (City Traffic Control)

• Descripción del Módulo

El módulo **EC_CTC (City Traffic Control)** es responsable de determinar la viabilidad del tráfico en la ciudad donde opera nuetro sistema EasyCab. Esta decisión se basa en la información climática obtenida desde el servicio externo OpenWeather. Si la temperatura en la ciudad es menor o igual a 0°C, el estado del tráfico se considera no viable ("KO"). En caso contrario, el tráfico es viable ("OK"). Este estado se proporciona mediante una API REST.

Adicionalmente, el módulo incluye una interfaz gráfica que permite cambiar dinámicamente la ciudad para la cual se evalúa el estado del tráfico.

• Interfaz y Funcionalidades

o API REST

1. **Ruta**: /trafico
    - **Método**: GET
    - **Descripción**: Devuelve el estado actual del tráfico y la ciudad configurada.
    - **Ejemplo de respuesta JSON**:

```
{
  "ciudad": "Benidorm",
  "estado_trafico": "OK"
}
```

o Interfaz Gráfica

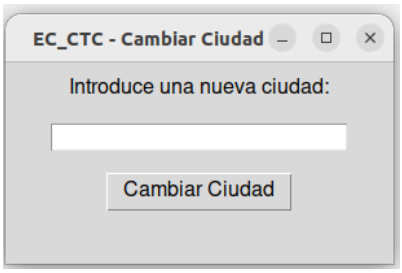
1. **Funcionalidad principal**: Permite al usuario cambiar la ciudad que se está evaluando.
2. **Elementos de la interfaz**:
    - Campo de entrada para el nombre de la ciudad.
    - Botón para confirmar el cambio de ciudad.
    - Mensajes de éxito o error al cambiar la ciudad.

• Detalles de Implementación

1. **Obtención del clima**:
    - El módulo utiliza la API de OpenWeather para obtener la información climática de la ciudad configurada.
    - URL de consulta: http://api.openweathermap.org/data/2.5/weather?q={ciudad}&appid={API_KEY}&units=metric
    - La temperatura se extrae del campo main.temp de la respuesta JSON.

2. **Lógica del estado del tráfico**:
    - Si la temperatura > 0°C: estado_trafico = "OK"
    - Si la temperatura <= 0°C o hay errores en la obtención de datos: estado_trafico = "KO"

3. **Interfaz Gráfica**:
    - Implementada con **Tkinter**.
    - Permite cambiar la ciudad configurada y notifica al usuario el éxito o error del cambio.



4. **Multitarea con hilos**:
    - Se utiliza **threading** para ejecutar simultáneamente:
        - La actualización periódica del estado del tráfico.
        - El servidor Flask para la API REST.
        - La interfaz gráfica.

○ EC_Registry

• Descripción del Módulo

El módulo **EC_Registry** es responsable de gestionar el alta y baja de los taxis en el sistema EasyCab. Utiliza una API REST para permitir que los taxis se registren o se desregistren. Además, implementa mecanismos de auditoría para registrar eventos relacionados con estas operaciones y garantiza la seguridad mediante la gestión de claves y tokens de autenticación.

• Interfaz y Funcionalidades

○ API REST

1. **Registrar Taxi**
    - **Ruta**: /register_taxi
    - **Método**: POST
    - **Descripción**: Registra un taxi en el sistema, actualizando su estado y posición si ya existe en la base de datos.
    - **Parámetros de entrada (JSON)**:
        - taxi_id (string, obligatorio): Identificador del taxi.
        - pos_x (int, opcional): Coordenada X inicial.
        - pos_y (int, opcional): Coordenada Y inicial.
    - **Ejemplo de respuesta exitosa**: { "mensaje": "Taxi 123 actualizado y está operativo" }
    - **Ejemplo de error**: { "error": "El taxi 123 ya está registrado y operativo" }

2. **Desregistrar Taxi**
    - **Ruta**: /unregister_taxi
    - **Método**: POST
    - **Descripción**: Elimina un taxi de la base de datos y de los archivos de gestión de claves y tokens.
    - **Parámetros de entrada (JSON)**:
        - taxi_id (string, obligatorio): Identificador del taxi.
    - **Ejemplo de respuesta exitosa**: { "mensaje": "Taxi 123 desconectado correctamente" }
    - **Ejemplo de error**: { "error": "El taxi 123 no existe en la base de datos" }
3. **Listar Taxis**
    - **Ruta**: /list_taxis
    - **Método**: GET
    - **Descripción**: Devuelve una lista de todos los taxis registrados y su estado actual.
    - **Ejemplo de respuesta**:
    - { "taxis": [{ "asignado": "SI", "operativo": "SI", "taxi_id": "123", "pos_x": 10, "pos_y": 15 }]}

• Detalles de Implementación

1. **Auditoría de Eventos**:
    - Todos los eventos importantes (registro, desregistro, errores) se registran en un archivo de auditoría (audit_log.txt).
    - Cada entrada incluye:
        - Fecha y hora del evento.
        - IP de origen.
        - Acción realizada.
        - Detalles del evento.
2. **Gestión de Archivos**:
    - **taxi.txt**: Almacena el estado y la posición de los taxis.
    - **token.txt**: Almacena los tokens de autenticación de los taxis.
    - **claves.txt**: Almacena las claves de seguridad asociadas a los taxis.

3. **Métodos de Seguridad**:
    - Gestión de claves y tokens para garantizar que solo taxis autorizados puedan operar.
    - Eliminación de datos sensibles al desregistrar un taxi.
4. **Flujo de Registro**:
    - Comprueba si el taxi ya existe en la base de datos.
    - Si está activo, actualiza su estado y posición.
    - Si no está registrado o está inactivo, devuelve un error adecuado.
5. **Flujo de Desregistro**:
    - Busca el taxi en la base de datos.
    - Elimina sus datos de los archivos correspondientes.
    - Actualiza su estado en taxi.txt a inactivo.

○ EC_API

• Descripción del Módulo

El módulo **EC_API** proporciona una API REST para exponer el estado del mapa de la ciudad, representando las posiciones de taxis, clientes y destinos. Esta información puede ser consumida por otros componentes del sistema, como el front-end, para mostrar visualmente el estado en tiempo real.

• Funcionalidades

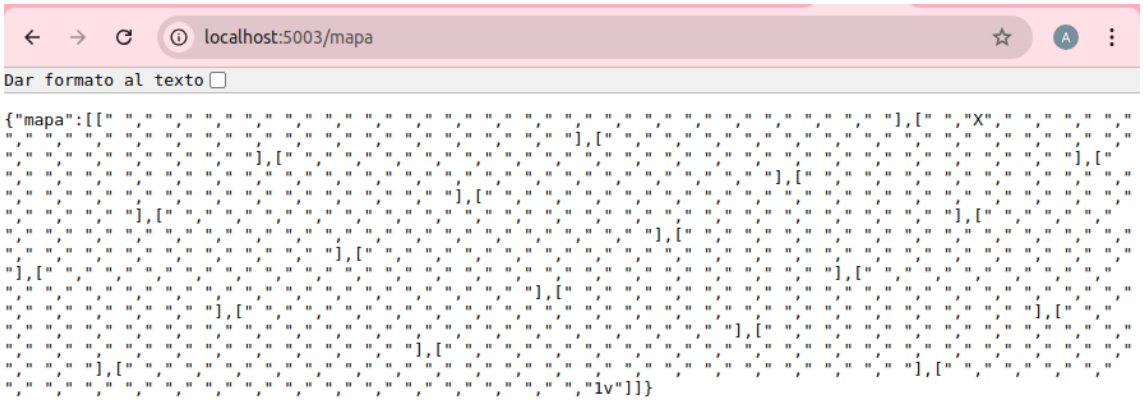
API REST

1. **Obtener Estado del Mapa**
    - **Ruta**: /mapa
    - **Método**: GET
    - **Descripción**: Devuelve el estado actual del mapa en forma de matriz 20x20, donde cada celda representa un espacio en la ciudad.
    - **Ejemplo de respuesta JSON**:
    - { "mapa": [[" ", " ", "T", " ", " "],
      ["C", " ", " ", " ", "D"],
      ...] }

- **Nota**: Los caracteres en la matriz pueden representar:
  - T: Taxi.
  - C: Cliente.
  - D: Destino.
  - " ": Espacio vacío.

• Detalles de Implementación

1. **Estado del Mapa**:
   - El mapa es una matriz global de 20x20 que representa la ciudad.
   - Inicialmente, todas las celdas están vacías (" ").
   - Puede ser actualizado mediante la función actualizar_mapa desde otros componentes, como **EC_Central**.
2. **Sincronización Concurrente**:
   - Se utiliza un threading.Lock para garantizar que las operaciones de lectura y escritura sobre el mapa sean seguras en entornos concurrentes.
3. **CORS**:
   - El módulo habilita **CORS** (Cross-Origin Resource Sharing) para permitir que otros dominios consuman los datos del API, especialmente el front-end.
4. **Interacción con EC_Central**:
   - La función actualizar_mapa permite que el estado del mapa sea modificado por **EC_Central** en tiempo real, reflejando cambios como nuevas posiciones de taxis o clientes.


localhost:5003/mapa
Dar formato al texto
{"mapa":[[" "," "],[" ","X"," "],[" "

○ FRONT-END

• Descripción del Módulo

El front-end del sistema EasyCab proporciona una interfaz visual para interactuar con los datos del sistema, incluyendo:

- El mapa en tiempo real que muestra la ubicación de taxis, clientes y destinos.
- Opciones para monitorear el estado y la operatividad de los taxis.
- Resumen de las solicitudes de viaje.

Este módulo está implementado utilizando **Astro** y componentes de **React**, y consume datos del API expuesto por el módulo EC_API.

• Interfaz y Funcionalidades



1. **Mapa Interactivo**:
   - Muestra un mapa en tiempo real con las posiciones de taxis, clientes y destinos.
   - Integra un iframe que consume directamente el recurso /mapa.html.
   - Actualiza la información en tiempo real mediante peticiones al API REST del módulo EC_API.
2. **Seguimiento de Taxis**:
   - Proporciona detalles sobre las ubicaciones y estados operativos de los taxis.
   - Acceso directo desde las tarjetas de la interfaz principal.
3. **Guía de despliegue**:
   - Ayuda para el despliegue.
4. **Documentación**:
   - Acceso a guías y descripciones de los módulos del sistema desde la interfaz.



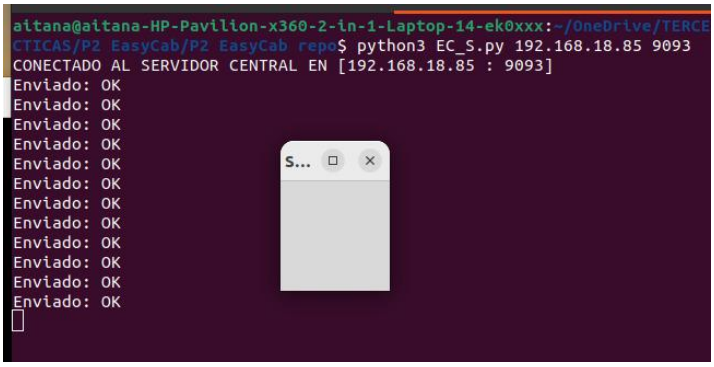
• Detalles de Implementación

1. **Diseño de la Interfaz**:
    - La interfaz principal se centra en un diseño modular con tarjetas informativas para cada funcionalidad clave.
    - Colores y gradientes personalizados para una experiencia de usuario intuitiva.
2. **Integración de Componentes**:
    - **Astro** gestiona la estructura del proyecto.
    - **React** permite interacciones dinámicas mediante componentes como RealTimeMap.
3. **Mapa en Tiempo Real**:
    - Implementado con un iframe que consume directamente el recurso /mapa.html.
    - La información del mapa es actualizada por el módulo EC_Central a través de EC_API.
    - **Detalles del archivo mapa.html**:
        - Representa el mapa como una cuadrícula de 20x20 utilizando un contenedor CSS flexible.
        - Celdas codificadas con clases específicas para mostrar:
            - Taxis (verde, rojo, amarillo dependiendo del estado).
            - Clientes (cliente).
            - Destinos (destino).
            - Espacios vacíos (fondo).
        - Obtiene datos mediante una función JavaScript que consulta el endpoint /mapa del EC_API cada 500ms.
        - Ajusta automáticamente el tamaño de las celdas según la resolución de la pantalla.
4. **Estilos y Experiencia del Usuario (UX)**:
    - Uso de CSS integrado en Astro para personalizar el diseño visual.
    - Efectos de hover y transiciones en las tarjetas para mejorar la interactividad.
5. **Accesibilidad y Compatibilidad**:
    - Compatible con navegadores modernos.
    - Soporte para diferentes resoluciones de pantalla.

○ SENSOR

• Interfaz

Este módulo utiliza una interfaz gráfica simple en Tkinter, que presenta una pequeña ventana para interactuar con el sistema mediante teclas (<s> para pausar o reanudar y <q> para finalizar la conexión con el Digital Engine).



• Detalles Implementación

- **Manejo de Eventos**: Configura eventos para sincronizar mensajes entre el hilo principal y el hilo de envío de datos, permitiendo enviar mensajes específicos (OK/KO/FIN) según la tecla presionada.

- **Hilo de Envío de Mensajes**: Un hilo secundario se encarga de enviar mensajes al Digital Engine a intervalos regulares o cuando se produce un cambio en el mensaje (por ejemplo, de OK a KO).

• Conexión Sockets

Este módulo se comunica con **EC_DE.py** únicamente a través de un socket TCP. Establece una conexión persistente y envía mensajes de control según el estado del sensor:

- **OK**: Movimiento permitido.
- **KO**: Movimiento detenido.
- **FIN**: Señal para cerrar la conexión y terminar el programa.

• Código

Funciones Principales:

- conectarDigitalEngine(): Establece la conexión con **EC_DE.py** mediante un socket TCP.
- enviarMensajes(): Hilo encargado de enviar mensajes periódicos al Digital Engine según el estado actual (OK o KO).
- teclaS_detectada(): Cambia el estado del mensaje entre OK y KO al presionar <s>.
- finalTransmision(): Envía un mensaje FIN y finaliza la conexión cuando se presiona <q>.

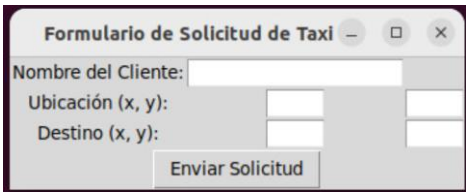
○ CUSTOMER

• Interfaz

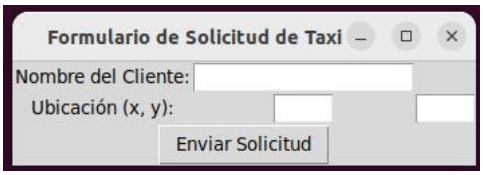
El archivo EC_Customer.py implementa una interfaz gráfica de usuario (GUI) en la que los clientes pueden solicitar un taxi y ver el estado de la solicitud. Para crear la interfaz, utiliza el módulo tkinter de Python, lo que le permite incluir elementos de entrada de texto para que los usuarios introduzcan su nombre, ubicación y destino. La interfaz también proporciona mensajes emergentes (messagebox) que notifican al usuario si su solicitud ha sido procesada, si el taxi está en camino o si ya ha llegado a su destino.

Tiene dos modalidades dependiendo de los comandos que se utilicen:

❖ Comando <-f>:
El cliente pone su nombre, ubicación y destino al que quiere dirigirse.



❖ Comando <nombre.json>:
El cliente escribe en el formulario su nombre y su ubicación. Los destinos a los que se quiere dirigir, se leerán del json pasado por línea de comandos.



• Detalles Implementación

La implementación de EC_Customer.py se basa en una arquitectura cliente-servidor, donde los clientes pueden enviar solicitudes de taxi a través de Kafka y recibir mensajes de estado en tiempo real de un servicio central. Los componentes clave incluyen:

1. **Kafka Producer**: Para enviar solicitudes de taxi a un topic de Kafka.
2. **Kafka Consumer**: Para recibir mensajes de estado desde el servicio central.
3. **Manejo de Estados**: Se asigna un identificador de cliente único (id_customer) que permite filtrar los mensajes relevantes para cada cliente. Además, se utiliza un campo codigoMensaje para diferenciar entre tipos de mensajes como confirmaciones, notificaciones de recogida y finalización del viaje.
4. **Recepción de Argumentos:** El módulo espera un parámetro de entrada que puede ser un formulario interactivo (-f) o un archivo JSON con ubicaciones de destino.

5. **Validación de Datos:** Se valida que los campos de ubicación y destino sean válidos y estén dentro de los límites del mapa. Si no, muestra un error.
6. **Función procesar_formulario:** Toma la entrada del usuario, valida los datos y, si son correctos, envía la solicitud a Kafka.
7. **Recepción de Mensajes de Kafka:** Un hilo dedicado escucha mensajes de Kafka para actualizar el estado de la solicitud, como el estado de recogida y finalización.
8. **Funciones auxiliares**: Se emplea json_serializer para serializar mensajes JSON antes de enviarlos, y recibirMensajesDeCentral para gestionar los mensajes entrantes del servidor central.

• Conexión Kafka- Sockets

La conexión entre EC_Customer.py y el servicio central es facilitada por Kafka, que actúa como middleware, permitiendo que los clientes publiquen mensajes y reciban actualizaciones de estado a través de topics. En este archivo, no se utilizan sockets directamente; en su lugar, los mensajes se envían y reciben a través de Kafka. La comunicación sigue un flujo asíncrono:

- El cliente envía una solicitud de taxi con **Kafka Producer**.
- El servicio central envía respuestas y actualizaciones de estado usando **Kafka Consumer**, que se configura con un group_id para mantener la sesión de cada

• Código

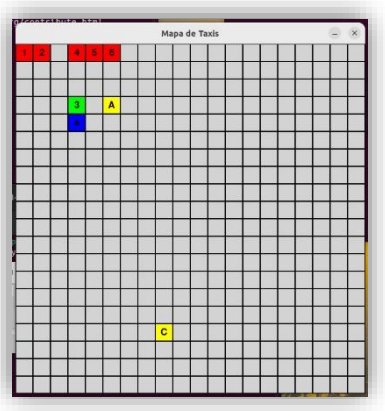
El módulo cuenta con las siguientes funciones principales:

1. **enviar_solicitud_taxi**: Crea un mensaje JSON con la solicitud del cliente y lo envía a Kafka. Envía una solicitud de taxi con los datos ingresados.
2. **recibirMensajesDeCentral**: Escucha actualizaciones del estado de la solicitud y muestra el estado del taxi a través de messagebox. Escucha las respuestas del servidor central y muestra mensajes al usuario.
3. **procesar_formulario**: Controla el flujo de envío de la solicitud al sistema y el inicio de la recepción de mensajes de Kafka. Llama a la función para enviar la solicitud.

Hilos: El módulo lanza un hilo para recibirMensajesDeCentral, permitiendo que la GUI siga activa mientras recibe actualizaciones del servidor.

○ MAPA PYGAME

• Interfaz



Este módulo utiliza Pygame para visualizar en tiempo real el estado de un mapa de taxis. Presenta una interfaz gráfica donde los taxis, clientes, y destinos se representan mediante celdas de diferentes colores. Las actualizaciones son continuas y reflejan los cambios en las posiciones de los elementos en el mapa.

• Detalles Implementación

- **Visualización en Pygame**: Define el tamaño de cada celda (TAMANO_CELDA), colores para los diferentes elementos (cliente, destino, taxi) y crea una cuadrícula que representa el mapa.

- **Actualización Asíncrona**: Usa una cola (queue.Queue) para manejar actualizaciones desde otros módulos, las cuales se procesan en el bucle principal de Pygame. Esta implementación asegura que los cambios en el mapa sean reflejados sin interferir con el flujo principal del programa.

• Conexión Kafka- Sockets

Aunque el módulo no interactúa directamente con Kafka o sockets, se integra en el sistema general permitiendo a los otros módulos enviar actualizaciones de posición a través de la función actualizar_mapa_pygame, que encola las actualizaciones para la visualización en Pygame.

• Código

Funciones Principales:

- **conectarDigitalEngine():** Establece la conexión con **EC_DE.py** mediante un socket TCP.
- **enviarMensajes():** Hilo encargado de enviar mensajes periódicos al Digital Engine según el estado actual (OK o KO).
- **teclas_detectada():** Cambia el estado del mensaje entre OK y KO al presionar <s>.
- **finalTransmision():** Envía un mensaje FIN y finaliza la conexión cuando se presiona <q>.
- **iniciar_pygame(ancho, alto)**: Configura la ventana de Pygame en función de las dimensiones del mapa (ancho y alto) y establece una fuente para el texto en las celdas.

- **dibujar_mapa(mapa_data)**: Dibuja el estado actual del mapa. Itera sobre las celdas de mapa_data y asigna un color y texto a cada celda en función de su contenido, diferenciando taxis, clientes y destinos.
- **actualizar_mapa_pygame(mapa_data)**: Recibe el estado del mapa desde otros módulos y lo encola para que main_pygame_loop lo procese.
- **main_pygame_loop()**: Ejecuta el bucle principal de Pygame. Procesa eventos de usuario (como cerrar la ventana) y extrae actualizaciones de la cola para redibujar el mapa.

○ GUÍA DE DESPLIEGUE

La guía de despliegue y sus comandos está hecha para Ubuntu, ya que es el sistema operativo que hemos utilizado para el desarrollo de nuestro proyecto

1. Desplegar KAFKA:

   Descargamos Kafka en nuestro equipo. Será necesario editar los valores listeners y advertisers del archivo server.properties:

   - Listeners: 0.0.0.0
   - Advertisers: Ip del equipo de despliegue.

   Abrimos dos terminales en el directorio de descarga y ejecutamos los siguientes comandos:

   - En Ubuntu:

     $ bin/zookeeper-server-start.sh config/zookeeper.properties

     $ bin/kafka-server-start.sh config/server.properties

2. Crear entorno virtual en Linux y activarlo:
   - Crear el entorno: *python3 -m venv venv*
   - Activarlo: *source venv/bin/actívate*

3. Instalar dependencias:

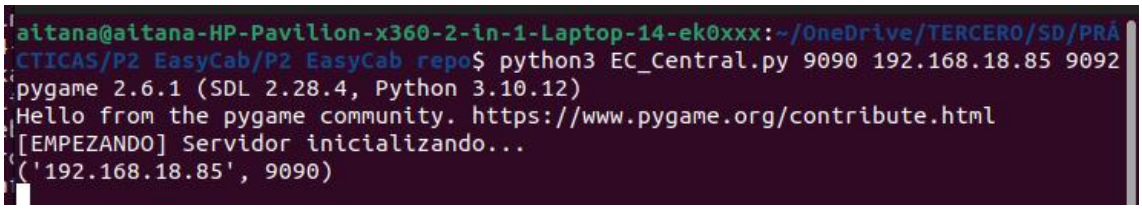
   pip install -r requirements.txt

4. Visualizar el front
    1. Nos situamos en la carpeta /front del proyecto: *cd /front*
    2. Ejecutamos el servidor de Astro: *npm run dev - -host*
    3. Abrimos el host que nos aparece en terminal.

5. Desplegar EC_CTC:
    - Ejecuta el comando con la clave de API de OpenWeather: python3 EC_CTC.py <archivo_API_key>
    - Por defecto, el servidor escuchará en http://<IP>:5000/trafico.

6. Desplegar EC_Registry: *python3 EC_Registry.py*
    - Por defecto, el servidor escuchará en http://<IP>:5001.

7. Desplegar Central:
    - Configura la conexión con Kafka, EC_CTC y el host del mapa en http://<IP>:5003/mapa.
    - Comando: *python3 EC_Central.py <Puerto escucha> <IP gestor> <Puerto gestor> <IP EC_CTC>*

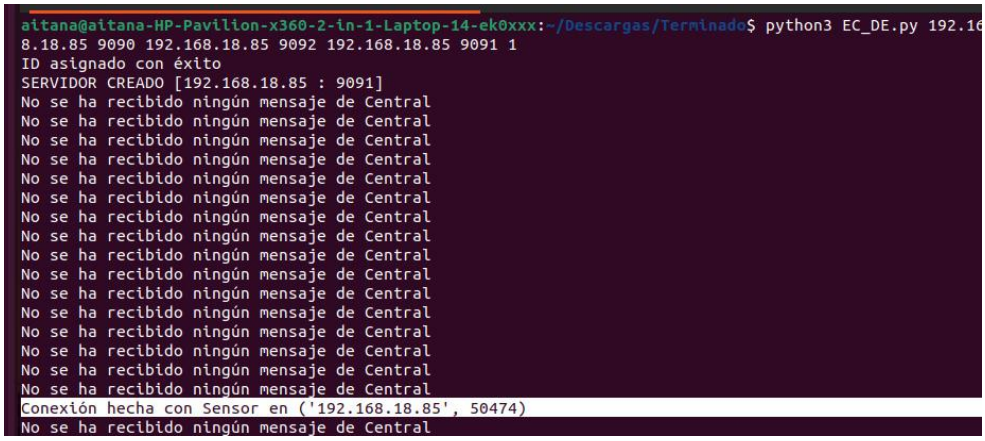


8. Dar de alta a los taxis:

Para dar de alta a un taxi se ha de ejecutar EC_DE.py y EC_S.py. Una vez ejecutado, presiona el botón "Conectar Taxi" en la interfaz del módulo EC_DE.

Comandos:

python3 EC_DE.py <IP central> <Puerto central> <IP gestor> <Puerto gestor> <IP Sensor> <Puerto Sensor> <ID Taxi>



python3 EC_S.py <IP DigitalEngine> <Puerto DigitalEngine>

```
aitana@aitana-HP-Pavilion-x360-2-in-1-Laptop-14-ek0xxx:~/Descargas/Terminado$ python3 EC_S.py 192.168
.18.85 9091
CONECTADO AL SERVIDOR CENTRAL EN [192.168.18.85 : 9091]
Enviado: OK
Enviado: OK
```

9. Añadir los customers:

   Tenemos dos opciones de uso:

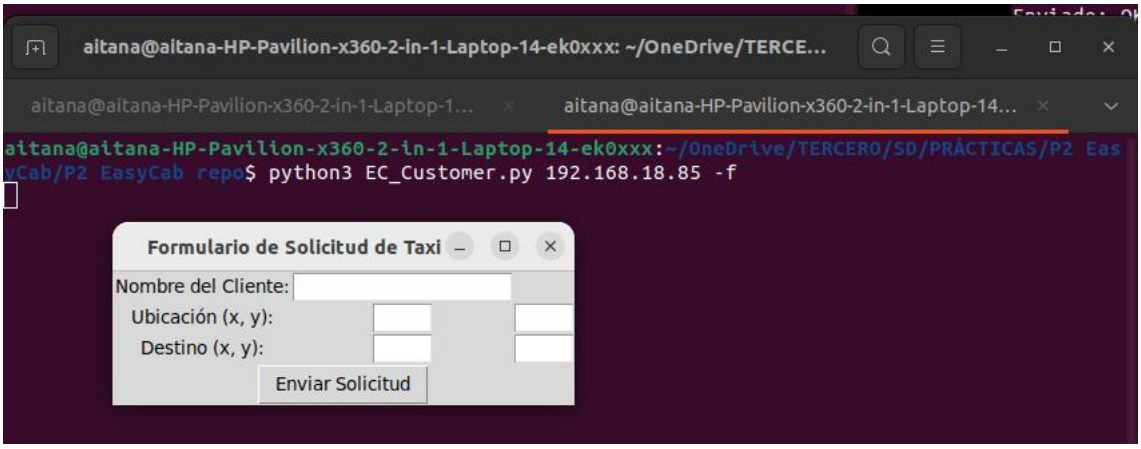
   - **Opción json:**

     Comando: python3 EC_Customer.py <IP_GESTOR> <nombre.json>



   - **Opción -f:**

     Comando: python3 EC_Customer.py <IP_GESTOR> <-f>



○ CONTROL DE VERSIONES

Para el control de versiones del proyecto, hemos utilizado Git y GitHub como herramientas principales. Cada miembro trabajó en una rama propia: *Aitana* y *Levan*, donde desarrollaban sus respectivas funcionalidades y realizaban commits individuales. Las ramas se fusionaban en la rama *develop*, que la utilizamos para integrar el código y realizar pruebas conjuntas. Una vez que las funcionalidades son probadas y aprobadas en *develop*, el código consolidado se fusiona en la rama *master*, la cual se utiliza para almacenar las versiones finales y estables del proyecto.